

HW2 HuggingFace - Yutang Lin 2300017741

This report only includes the problems that needs to write answers.

Task 2

I ran three experiment for each model, with same learning rate and other parameter configurations. The training and evaluation curve are as follows:



The final evaluation result is:

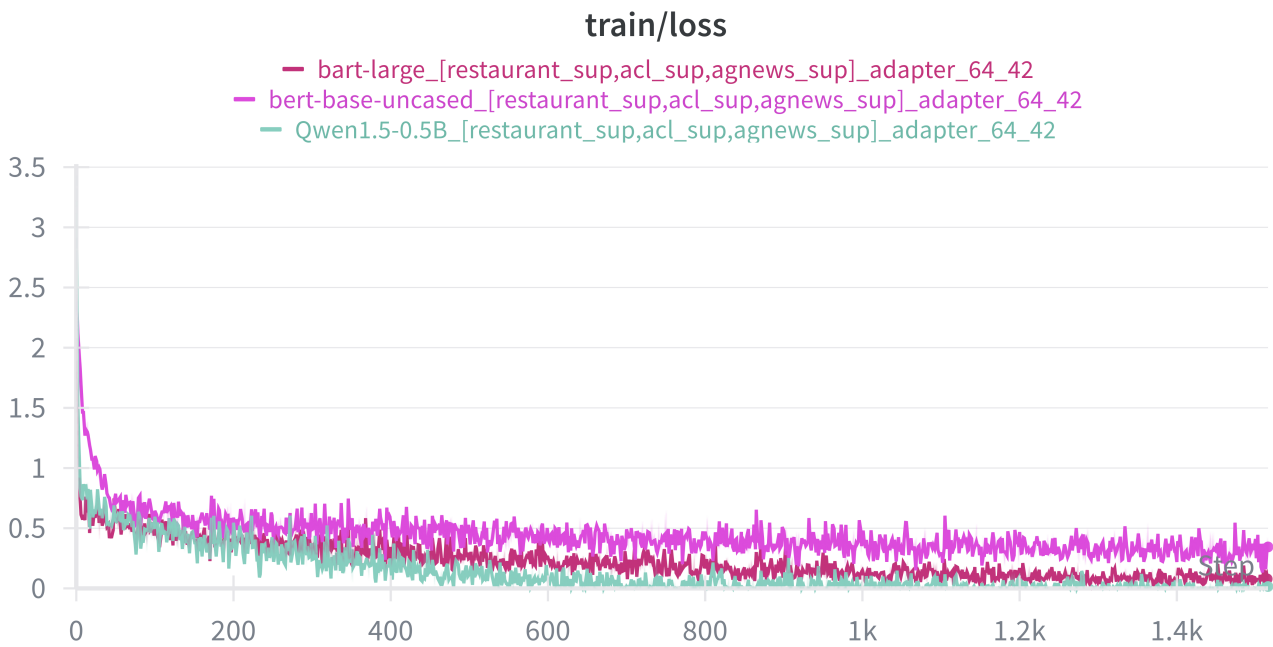
Model	Metric	Mean	Std
BART (facebook/bart-large)	Accuracy	0.884101	0.001070

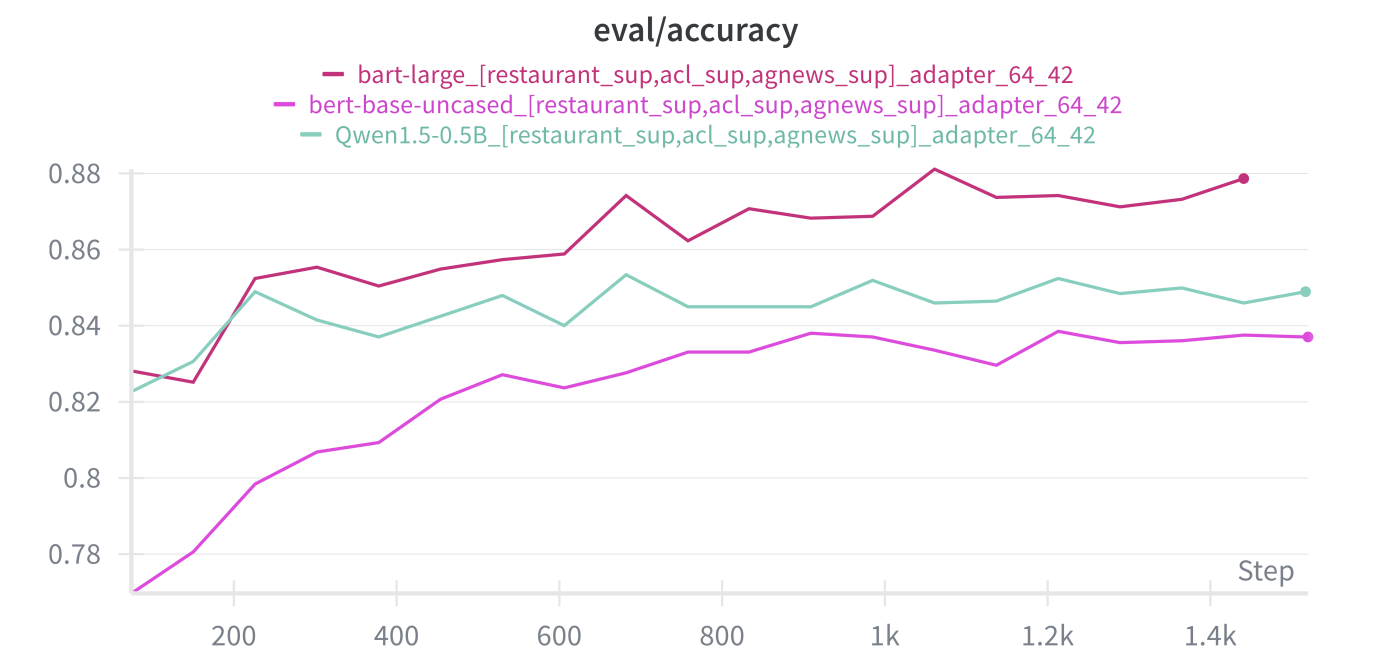
Model	Metric	Mean	Std
(3 runs)	Macro F1	0.782596	0.012656
	Micro F1	0.884101	0.001070
	Weighted F1	0.877161	0.001857
BERT (bert-base-uncased)	Accuracy	0.851577	0.002035
(3 runs)	Macro F1	0.745521	0.020034
	Micro F1	0.851577	0.002035
	Weighted F1	0.846690	0.002494
Qwen (Qwen1.5-0.5B)	Accuracy	0.858511	0.003928
(3 runs)	Macro F1	0.758282	0.015354
	Micro F1	0.858511	0.003928
	Weighted F1	0.849221	0.004103

Task 3

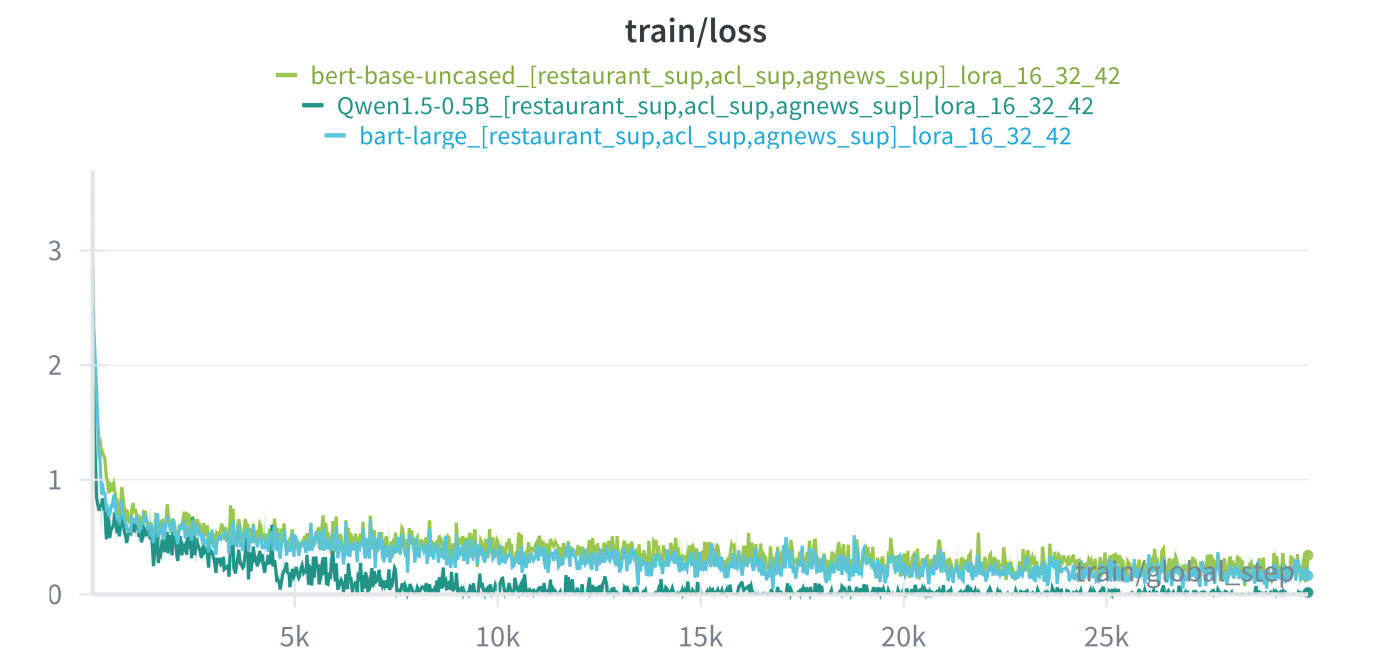
PEFT Curves

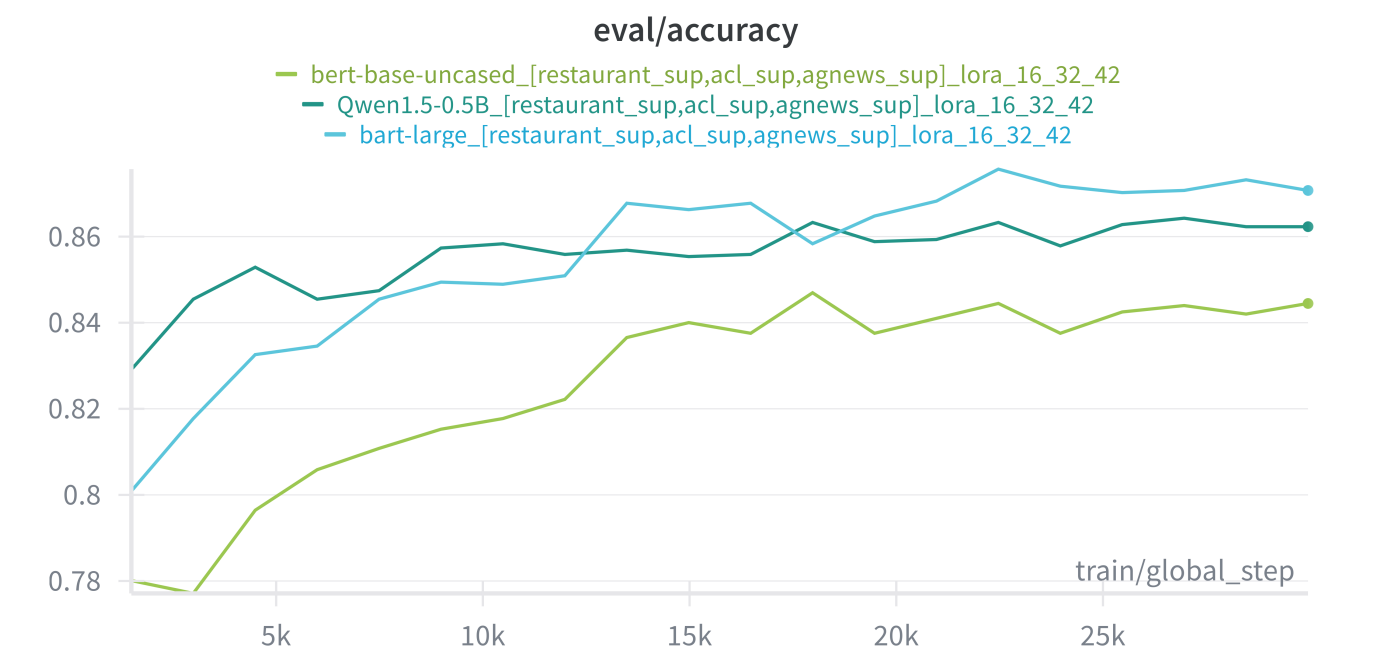
Adapter's curves:



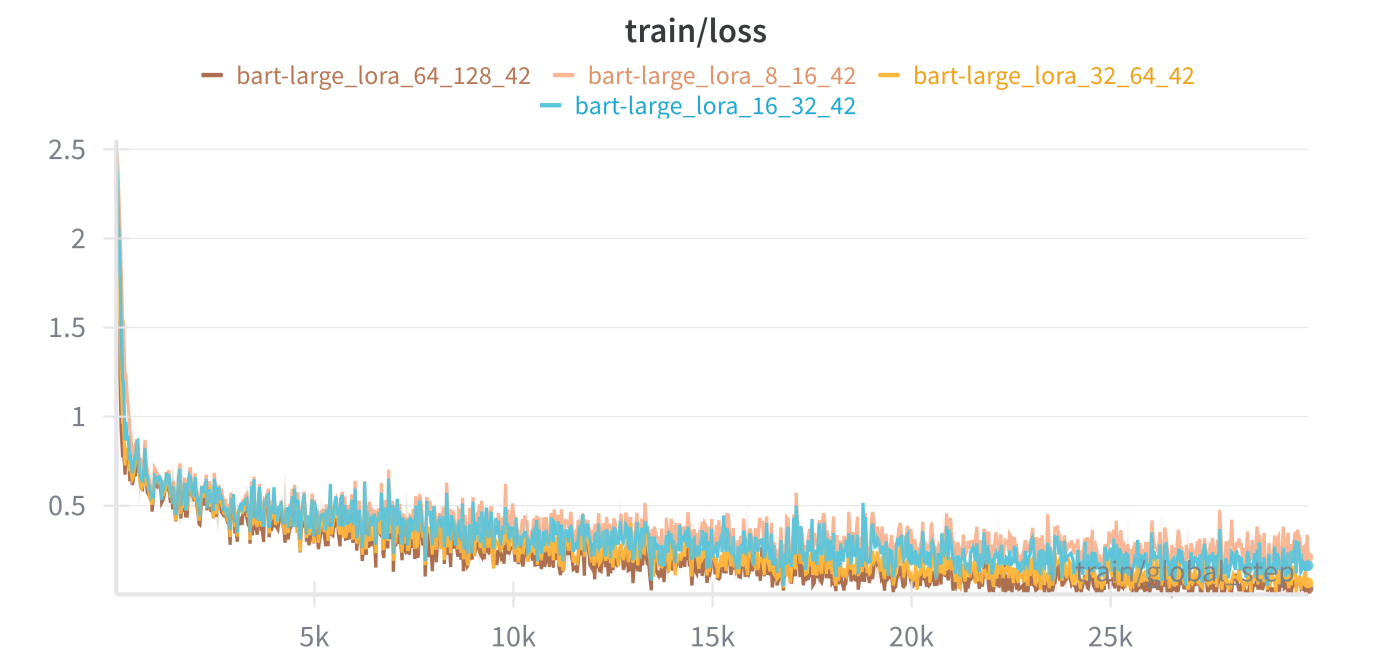


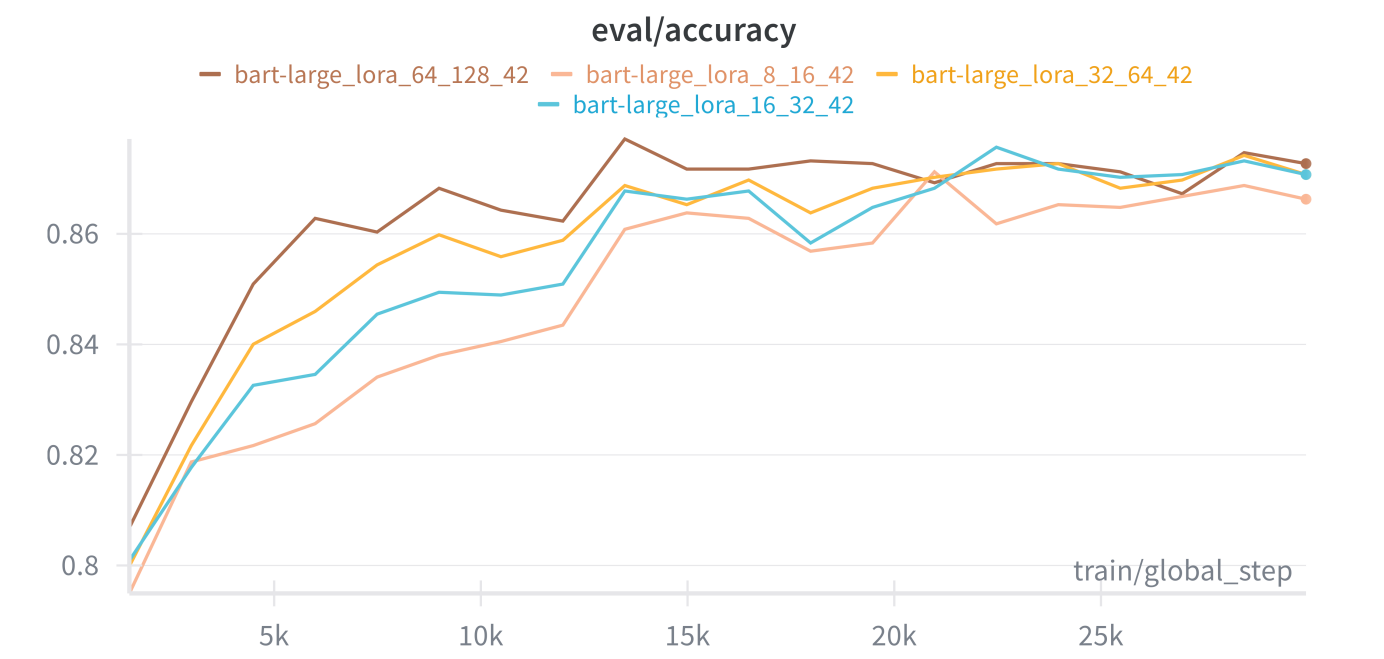
LoRA's curves:





LoRA's ablation on rank:





PEFT Results by Model and Method

Model	Method	Configuration	Accuracy	Macro F1	Micro F1	Weighted F1
BART	LoRA	rank=16, alpha=32	0.870728	0.752369	0.870728	0.861363
BART	Adapter	size=64	0.876176	0.774592	0.876176	0.868079
BERT	LoRA	rank=16, alpha=32	0.844477	0.651129	0.844477	0.834313
BERT	Adapter	size=64	0.837048	0.599638	0.837048	0.822875
Qwen	LoRA	rank=16, alpha=32	0.862308	0.777807	0.862308	0.854469
Qwen	Adapter	size=64	0.848935	0.753391	0.848935	0.842043

LoRA Rank Ablation Study (BART)

Rank	Alpha	Accuracy	Macro F1	Micro F1	Weighted F1
8	16	0.866270	0.709840	0.866270	0.855158
16	32	0.872709	0.753648	0.872709	0.864567
32	64	0.870728	0.778185	0.870728	0.864404
64	128	0.872709	0.774705	0.872709	0.865881

VRAM Requirements and PEFT Analysis

1. How much VRAM do you need to full fine-tune a Llama-3-70B?

For full fine-tuning of Llama-3-70B, the VRAM requirements are:

- **Model weights** (fp16/bf16): $70B \times 2 \text{ bytes} = 140 \text{ GB}$

- **Optimizer states** (Adam optimizer): $2 \times \text{model size} = \mathbf{280\ GB}$
 - Momentum states: $70\text{B} \times 2\ \text{bytes} = 140\ \text{GB}$
 - Variance states: $70\text{B} \times 2\ \text{bytes} = 140\ \text{GB}$
- **Gradients**: Same as model size = **140 GB**
- **Activations**: Depends on batch size and sequence length, typically **50-200 GB** for reasonable batch sizes

Total estimate: ~610-760 GB of VRAM for full fine-tuning.

2. How much VRAM is saved when using PEFT methods?

LoRA reduce VRAM requirements:

- **Base model weights** (frozen, fp16): **140 GB**
- **LoRA parameters** (rank=16, typical): $\sim 0.1\text{-}0.5\%$ of model size
 - For 70B model: $\sim 70\text{M parameters} \times 2\ \text{bytes} = \sim \mathbf{140\ MB}$
- **Optimizer states for LoRA**: $2 \times \text{LoRA size} = \sim \mathbf{280\ MB}$
- **Gradients for LoRA**: **~140 MB**
- **Activations**: Similar to full fine-tuning, but can use smaller batches = **~50-200 GB**.

Total estimate: ~190-340 GB of VRAM (mostly from base model + activations).

VRAM savings: ~220-280 GB (approximately **40-50% reduction**).

3. Can PEFT speed up the training process? Can it speed up the reasoning process?

Training Speed:

- **PEFT can speed up training** because:
 1. **Fewer parameters to update**: Only 0.1-1% of parameters need gradient computation and optimizer updates, reducing computation per step
 2. **Faster optimizer steps**: Adam optimizer only updates LoRA parameters (millions instead of billions), making each optimization step much faster
 3. **Better memory efficiency**: Can use larger batch sizes or longer sequences within the same VRAM budget, potentially improving training throughput
 4. **Reduced communication overhead**: In distributed training, only LoRA weights need to be synchronized, not the entire model

Reasoning/Inference Speed:

- **PEFT does not speed up inference** because:
 1. **Full model still loaded**: The base model (140 GB) must still be loaded into memory for inference
 2. **LoRA weights merged**: During inference, LoRA weights are merged into the base model, resulting in the same computational graph as the full model

Task 4

DeepSpeed Differences

Stage	Parameters	Gradients	Optimizer States	Memory Savings	Communication Overhead
0	Full copy	Full copy	Full copy	1x (baseline)	Low (all-reduce)
1	Full copy	Full copy	Partitioned	~4x	Medium
2	Full copy	Partitioned	Partitioned	~8x	Medium-High
3	Partitioned	Partitioned	Partitioned	~Nx (N=GPUs)	High (gather on-demand)

Trade-offs:

- **Stage 0 to Stage 2:** Increasing memory savings with moderate communication overhead
- **Stage 2 to Stage 3:** Maximum memory savings but requires parameter gathering during forward/backward passes, which can slow down training
- **Offloads:** Further memory savings at the cost of CPU-GPU data transfer overhead

Yaml for Fine-Tuning

The files are available at `qwen1_5_lora.yaml` and `qwen1_5_lora_merge.yaml`

Fine-Tuning and Evaluation Results

Training curves:





Evaluation results using `eval.py`:

```
Evaluation Results: {'BLEU': 0.00634119782176934, 'ROUGE-L': 0.14857494937905855, 'BERTScore': 0.4891209602355957}
```